

Mockito Hands-On Exercises

Exercise 1: Mocking and Stubbing

Scenario:

You need to test a service that depends on an external API. Use Mockito to mock the external API and stub its methods.

Steps:

1. Create a mock object for the external API.
2. Stub the methods to return predefined values.
3. Write a test case that uses the mock object.

Solution:

```
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-api</artifactId>
    <scope>test</scope>
  </dependency>
  <!-- Optionally: parameterized tests support -->
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-params</artifactId>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-core</artifactId>
    <version>5.12.0</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

ExternalApi.java

```
package com.example;

public interface ExternalApi {
    String getData();
}
```

MyService.java

```
package com.example;

public class MyService {

    private ExternalApi api;

    public MyService(ExternalApi api) {
        this.api = api;
    }

    public String fetchData() {
        return api.getData();
    }
}
```

MyServiceTest.java

```
package com.example;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.when;

import org.junit.jupiter.api.Test;
import org.mockito.Mockito;

public class MyServiceTest {

    @Test
    void testExternalApi() {

        ExternalApi mockApi = Mockito.mock(ExternalApi.class);

        when(mockApi.getData()).thenReturn("Mock Data");

        MyService service = new MyService(mockApi);

        String result = service.fetchData();

        System.out.println("Fetched Data: " + result);

        assertEquals("Mock Data", result);
    }
}
```

Output:

The screenshot displays an IDE with the following components:

- Project Explorer:** Shows the project structure with a package `com.example` containing `MyServiceTest` and `testExternalApi0 - void`.
- JUnit Runner:** Indicates the test `MyServiceTest (Runner: JUnit5) [1.225 s]` passed after 1.334 seconds. It shows 1/1 runs, 0 errors, and 0 failures.
- Source Editor:** Contains the following Java code:

```
1 package com.example;
2
3 import static org.junit.jupiter.api.Assertions.assertEquals;
4 import static org.mockito.Mockito.when;
5
6 import org.junit.jupiter.api.Test;
7 import org.mockito.Mockito;
8
9 public class MyServiceTest {
10
11     @Test
12     void testExternalApi() {
13         ExternalApi mockApi = Mockito.mock(ExternalApi.class);
14         when(mockApi.getData()).thenReturn("Mock Data");
15
16         MyService service = new MyService(mockApi);
17         String result = service.fetchData();
18         System.out.println("Fetched Data: " + result);
19         assertEquals("Mock Data", result);
20     }
21 }
```
- Console:** Shows the output of the test run:

```
<terminated> MyServiceTest [JUnit] C:\Program Files\Java\jdk-17\bin\javaw.exe (29 Jun 2026, 11:44:29 pm - 11:44:31 pm elapsed 0.0001) [pid: 11972]
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
Fetched Data: Mock Data
```

Exercise 2: Verifying Interactions

Scenario:

You need to ensure that a method is called with specific arguments.

Steps:

1. Create a mock object.
2. Call the method with specific arguments.
3. Verify the interaction.

Solution:

EmailService.java

```
package com.example;

public interface EmailService {
    void sendEmail(String email);
}
```

UserService.java

```
package com.example;

public class UserService {

    private EmailService emailService;

    public UserService(EmailService emailService) {
        this.emailService = emailService;
    }

    public void registerUser(String email) {
        emailService.sendEmail(email);
    }
}
```

UserServiceTest.java

```
package com.example;

import static org.mockito.Mockito.mock;
import static org.mockito.Mockito.verify;

import org.junit.jupiter.api.Test;

public class UserServiceTest {

    @Test
    void testRegisterUser() {

        EmailService mockEmailService = mock(EmailService.class);

        UserService userService = new UserService(mockEmailService);

        userService.registerUser("saiabhiram@example.com");

        verify(mockEmailService).sendEmail("saiabhiram@example.com");
    }
}
```

Output:

